

Wheat Water Productivity Dashboard

Project overview, setup and API reference

Ethiopian Institute of Agricultural Research · IWMI East Africa · FAO WaPOR Phase II
Generated 2026-08-19

Wheat Water Productivity (WWP) Dashboard

Interactive web dashboard that assesses and monitors wheat water productivity in Ethiopia from FAO WaPOR v3 satellite data.

Built for the Ethiopian Institute of Agricultural Research (EIAR) under the WaPOR Phase II project, developed by IWMI East Africa with FAO, supported by the Government of the Netherlands. Implements the deliverables in [ToR_Dashboard_Developer.docx](#) ; see the [ToR Coverage](#) document for the work-package mapping and the [Architecture](#) document for the system design.

What it does

Pick an area of interest three ways — an administrative unit (region → zone → woreda), an uploaded field or scheme boundary (zipped shapefile or GeoJSON), or a polygon drawn on the map. Choose a production system (rainfed or irrigated) and a season, then run the analysis. The backend resolves the season to explicit SOS and EOS dates, retrieves seasonal WaPOR NPP and AETI over that window, and converts them to wheat biomass, grain yield and water productivity. It returns a water-productivity raster with zonal statistics, a class distribution, a five-season trend, and the estimation chain behind the area mean. Click any pixel for its value, then **Show the derivation** to see every input, parameter and intermediate that produced it.

Upload a scheme file instead and the dashboard switches to the reference notebook's own workflow. A shapefile or GeoJSON whose features carry `ID`, `SOS` and `EOS` (plus `Location` for sample points) is estimated feature by feature, each over its own growing season, and the results come back the way the notebook writes them: `NPP`, `EYield_tpha`, `AETI_mm`, `WP_kgpm3` and `LGP` appended to the attributes the file arrived with, point samples collapsed to one row per plot by median, and the two figures the notebook plots (estimated yield and water productivity per scheme). Both tables download as CSV with those columns.

The Upload panel also lists ready-made files, so the workflow can be seen without preparing one. Where the machine holds the 2026 campaign shapefiles — in `./Data`, or wherever `WWP_CAMPAIGN_DATA` points — it offers the two the reference notebook reads, `Irrigated_Wheat_2026` (6 scheme boundaries) and `Irrigated_Wheat_2026_PNT` (57 sample points). Where it does not, and on any public deployment, it offers a generated stand-in in the same schema instead: those shapefiles are IWMI field data and are not part of this repository. Either way the file is loaded through the ordinary upload path, so what is tried is the real journey.

The method

The estimate is deterministic — no statistical model, no fitted coefficients:

```
TB = AOT · fc · NPP · 22.222 / (1 - mc)    total biomass, kg DM/ha
Y   = TB · hi                             grain yield, kg/ha
CWP = Y / SWC,   SWC = AETI · 10          water productivity, kg/m³
```

This is a port of the IWMI reference notebook `ETH_WWP_WaPORv3_Irrigaed_Wheat_2026_Final.ipynb` (`etwapor.productivity.estimate_wheat_wp`), which remains the authoritative implementation. `tests/test_notebook_parity.py` replays every result that notebook publishes — 6 irrigation-scheme polygons and 10 point samples — through this code and requires the same yield and water productivity at the notebook's own precision. The check also gates the Docker build.

Parity covers the arithmetic, which is what the notebook's published records pin down. It does not extend to the retrieval: the reference's URL builder resolves all three dekads of a month to the month's first raster, so live WaPOR retrieval here yields different — and correct — seasonal NPP and AETI.

`backend/app/wapor.py` and `docs/ARCHITECTURE.md` set out the evidence; it is worth raising with IWMI before the campaign figures are published.

Crop parameters come from `etwapor.data.wheat`, the reference implementation's own values: AOT 0.85, fc 0.90, mc 0.15, hi 0.48 (FAO, 2020b). They are overridable without touching code — see `backend/app/wwpt.py` and `WWP_CROP_PARAMS`. Replacing them with EIAR-derived, agro-ecology-specific values is the intended next step, as the reference notebook recommends.

Requirements

- Python 3.10+
- Node.js 18+

Quick start

```
# 1. Backend
cd backend
pip install -r requirements.txt
uvicorn app.main:app --port 8000

# 2. Frontend - in a second terminal
cd frontend
npm install
npm run dev                                # http://localhost:5173, proxies /api to :8000
```

For a single-process deployment, build the frontend and let the backend serve it:

```
cd frontend
npm run build                # emits frontend/dist
cd ../backend
uvicorn app.main:app --port 8000
# dashboard at http://localhost:8000, API at http://localhost:8000/api
```

One line per command rather than `cmd1 && cmd2`: Windows PowerShell 5.1 has no `&&` and fails to parse it. Chain with `;` there. Add `--reload` while developing — without it uvicorn keeps serving the code it started with, and a backend change appears to have had no effect. Frontend changes always need `npm run build` (or `npm run dev` on :5173, which proxies `/api` to :8000).

`npm install` may report that esbuild's install script was skipped. If the build then fails, run `npm approve-scripts esbuild` once.

Data sources

The service starts on a **synthetic provider** (`geodata.SyntheticProvider`) that generates deterministic, spatially coherent NPP and AETI fields, so the whole pipeline runs without network access. Results produced this way are labelled *Demonstration data* in the results panel and the footer, and flagged `"synthetic": true` in every API response — the method is real in both modes, the inputs are not.

For live FAO WaPOR v3 retrieval:

```
pip install rasterio        # GDAL; not a default dependency
WWP_PROVIDER=wapor uvicorn app.main:app --port 8000
curl localhost:8000/api/wapor/check # verify catalogue, units and access first
```

`backend/app/wapor.py` sums dekadal NPP and AETI over the season window, reading only the needed tiles from the published cloud-optimised GeoTIFFs. It reads the **L2 national mapsets at 100 m** (`L2-NPP-D`, `L2-AETI-D`), the level the reference implementation uses; set `WWP_WAPOR_LEVEL=L1` for the 300 m global products, or `L3` with `WWP_WAPOR_SCHEME=KOG|AWH` for the 20 m scheme mosaics.

`/api/wapor/check` verifies that contract on demand — raster present, unit and scale as assumed — and is the first thing to run in a new deployment.

Project layout

backend/	
app/	
main.py	FastAPI app; mounts /api and serves frontend/dist
api.py	REST routes, request validation
wwpt.py	The estimation method: equations, crop parameters, seasons
engine.py	Analytical engine: grid, raster, statistics, chain, export
geodata.py	Provider interface + synthetic NPP/AETI provider
wapor.py	FAO WaPOR v3 retrieval (catalogue, COG sampling)
schemes.py	Per-feature scheme analysis: the notebook's input and output
aoi.py	AOI resolution, shapefile/GeoJSON validation, geometry
cache.py	TTL response cache + idempotency-key store
pnglib.py	Dependency-free RGBA PNG encoder for the result raster
frontend/	
src/	
App.jsx	Dashboard shell, state, user journeys
MapView.jsx	Leaflet map: AOI, raster overlay, drawing, pixel inspect
charts.jsx	Inline-SVG charts and the estimation-chain view
schemes.jsx	Per-scheme results: notebook tables and the two figures
content.jsx	Methodology, data sources, user guide, citation, disclaimer
api.js	Backend client with typed error messages
styles.css	EIAR visual identity and layout
tests/	
test_notebook_parity.py	Reproduces the reference notebook's arithmetic (no server)
test_campaign_files.py	Ingest parity on the real 2026 campaign shapefiles
test_api_e2e.py	55 API checks: journeys, caching, validation, errors
test_schemes.py	44 checks: per-scheme input, output, CSV, validation
test_ui.mjs	65 browser checks: rendering, interaction, layout
docs/	
ARCHITECTURE.md	System design, data flow, API reference
DEPLOYMENT.md	Railway deployment, staging → production, env vars
TOR_COVERAGE.md	Work-package coverage and what deployment still needs
build_pdf.mjs	Renders the documentation to docs/pdf/*.pdf
Dockerfile	Two-stage build; parity check gates the image
railway.json	Railway builder, health check, single-replica pin

Deploying

`Dockerfile` builds the whole application — frontend bundle and backend — runs the notebook parity check, and binds `$PORT`. See the [Deployment Guide](#) for the Railway staging-then-production walkthrough and the environment variables.

API

All routes are under `/api`. Interactive documentation is generated at `/docs` when the server is running.

METHOD	ROUTE	PURPOSE
GET	<code>/health</code>	Liveness plus the active data provider
GET	<code>/admin-units</code>	Region/zone/woreda tree, valid years and seasons
POST	<code>/upload</code>	Validate a boundary file, return its id, bounds and GeoJSON
POST	<code>/analysis</code>	Run an analysis over an AOI; honours <code>Idempotency-Key</code>
POST	<code>/schemes/analysis</code>	Estimate every feature in an uploaded scheme file, each over its own SOS/EOS
GET	<code>/schemes/export/csv</code>	Per-feature or per-plot results, in the notebook's columns
GET	<code>/schemes/datasets</code>	Scheme files this deployment can load: campaign files where present, plus a generated sample
GET	<code>/schemes/dataset</code>	One of those files as GeoJSON, ready to post back to <code>/upload</code>
GET	<code>/schemes/sample</code>	The generated sample on its own
GET	<code>/predict</code>	WWP at a single point
GET	<code>/explain</code>	Full derivation for one point: inputs, parameters, steps
GET	<code>/export/csv</code>	Per-cell values for a completed run
GET	<code>/method</code>	Equations, crop parameters, units, provider, resolution
GET	<code>/wapor/check</code>	Verify WaPOR v3 catalogue access, units and scaling

Environment variables

VARIABLE	DEFAULT	PURPOSE
<code>WWP_PROVIDER</code>	<code>synthetic</code>	<code>synthetic</code> or <code>wapor</code>
<code>WWP_CROP_PARAMS</code>	—	Path to a JSON file overriding AOT, fc, mc, hi
<code>WWP_WAPOR_LEVEL</code>	<code>L2</code>	WaPOR spatial level: <code>L2</code> is 100 m national, as the reference reads
<code>WWP_WAPOR_SCHEME</code>	—	Scheme code for L3 mosaics (<code>KOG</code> , <code>AWH</code>)
<code>WWP_WAPOR_BASE</code>	FAO GISMGR	Catalogue base URL
<code>WWP_CAMPAIGN_DATA</code>	<code>./Data</code>	Directory holding the 2026 campaign shapefiles
<code>WWP_CORS_ORIGINS</code>	Vite dev server	Comma-separated CORS allowlist

Tests

```
python tests/test_notebook_parity.py    # no server, no network
python tests/test_campaign_files.py     # skips unless the campaign shapefiles are present

# start the backend, then:
python tests/test_api_e2e.py            # 55 API checks
python tests/test_schemes.py            # 44 per-scheme checks

cd tests
npm install playwright && npx playwright install chromium
node test_ui.mjs                        # 65 browser checks, writes screenshots/
```

Point any suite at a different host with `WWP_BASE=http://host:port` — including a deployed staging URL, which makes the API suite a real smoke test rather than a liveness ping.

Known limitations

- Crop parameters are general FAO values, not Ethiopia-specific. They reproduce the reference notebook, but neither has been validated against field-measured yields.
- The season selector applies one SOS/EOS window to the whole area of interest. The reference notebook works per plot, with each feature carrying its own dates.
- `backend/app/wapor.py` has been verified against the live catalogue for its raster-code convention, download URLs, units and scale factors, but its pixel reads have not been executed — that needs `rasterio`. Run `/api/wapor/check` before relying on it.